1. File Concept

- A **file** is a named collection of related information recorded on secondary storage.
- Types of files: Text files, binary files, program files, multimedia files.
- Attributes of a file:
    - Name
    - Identifier
    - Type
    - Location
    - Size
    - Protection (read, write, execute permissions)
    - Time/date of creation and modification.

---

2. Access Methods

Access methods define how files are accessed. Common types include:

1. **Sequential Access**:
    - Data is accessed in a linear manner.
    - Operations: `read_next` or `write_next`.
    - Example: Text files.
2. **Direct Access**:
    - Allows access to any block directly using a block number or address.
    - Example: Database systems.
3. **Indexed Access**:
    - Uses an index to locate files.
    - Allows random access using pointers stored in an index table.
    - Example: ISAM (Indexed Sequential Access Method).

---

3. Directory Structure

A directory contains information about files and may be organized in several ways:

1. **Single-level directory**:
    - One directory for all users.
    - Simple but lacks scalability.
2. **Two-level directory**:
    - Separate directories for each user.
    - Prevents name collisions.
3. **Tree-structured directories**:
    - Hierarchical organization.
    - Supports subdirectories.
4. **Acyclic graph directories**:
    - Allows shared files/directories without creating cycles.
5. **General graph directory**:
    - Allows more complex structures but requires cycle detection.

---

4. File System Mounting

- The process of associating a file system with a directory.
- Steps:
    - Specify the device and mount point.
    - Validate the file system (check integrity).
    - Add the file system to the global namespace.

---

5. File Sharing

- Enables multiple users/processes to access files simultaneously.
- Modes:
    - Read-Only.
    - Read-Write.
- Mechanisms:
    - Shared access through locking (mandatory or advisory).
    - Network file sharing protocols (e.g., NFS, SMB).

---

6. Protection

- File protection mechanisms ensure that unauthorized users cannot access or modify files.
- Techniques:
    - **Access Control Lists (ACLs)**: Specify users and their permissions.
    - **Access Rights**: Read, Write, Execute, Delete.
    - **Encryption**: Protect file contents during storage and transmission.

---

7. File System Structure

- Comprises:
    - Boot control block: Contains booting information.
    - Volume control block: Information about file system layout.
    - Directory structure: Maintains metadata about files.
    - File control blocks (FCBs): Stores file details (permissions, size, etc.).

---

8. File System Implementation

1. **File Control Block (FCB)**:
    - Contains metadata like size, permissions, timestamps, and location.

2. **Logical vs. Physical Structure**:
    - Logical file systems: User-level operations like naming and permissions.
    - Physical file systems: Disk-level operations.

---

9. Directory Implementation

- Two common approaches:
    1. **Linear List**:
        - Files are stored as a list.
        - Simple but slow for searches.
    2. **Hash Table**:
        - Maps file names to directory entries.
        - Faster but requires handling collisions.

---

10. Allocation Methods

Describes how disk blocks are allocated to files:

1. **Contiguous Allocation**:
    - Files occupy consecutive blocks.
    - Fast but may lead to fragmentation.
2. **Linked Allocation**:
    - Blocks linked via pointers.
    - No fragmentation but slower random access.
3. **Indexed Allocation**:
    - Index table stores pointers to file blocks.
    - Efficient but requires extra space for the index.

---

11. Free Space Management

Techniques to manage unallocated disk space:

1. **Bit Vector**:
    - A bit array where 0 = free and 1 = allocated.
    - Efficient but requires space for the bitmap.
2. **Linked List**:
    - Free blocks linked in a list.
    - Simple but slower for large disks.
3. **Grouping**:
    - Groups pointers to free blocks in a block.
4. **Counting**:
    - Tracks contiguous blocks of free space.

---

12. Kernel I/O Subsystems

- Manages I/O operations efficiently.
- Includes:
    - **Buffering**: Temporary storage to handle speed mismatches.
    - **Caching**: Retains frequently accessed data in memory.
    - **Spooling**: Handles multiple simultaneous I/O requests.
    - **Device Drivers**: Interfaces between the OS and hardware.

---

13. Disk Structure

- Logical structure: Tracks (circular), sectors (segments), and cylinders (stacked tracks).
- Physical structure:
    - Platters, read/write heads, and spindle.

---

14. Disk Scheduling

- Determines the order of I/O requests to improve performance.

1. **FCFS (First Come First Serve)**:
    - Simple but inefficient.
2. **SSTF (Shortest Seek Time First)**:
    - Prioritizes nearest requests.
3. **SCAN (Elevator Algorithm)**:
    - Moves across the disk and reverses direction.
4. **C-SCAN**:
    - Always moves in one direction and starts over.
5. **LOOK/C-LOOK**:
    - Optimized SCAN/C-SCAN.

---

15. Disk Management

- Handles low-level disk operations:
    - Formatting: Dividing a disk into sectors.
    - Partitioning: Splitting the disk into logical units.
    - RAID: Redundant Array of Independent Disks for performance and redundancy.

---

16. Swap Space Management

- A dedicated space on disk for virtual memory.

- Functions:
  - Stores processes temporarily when RAM is full.
  - Improves memory utilization.
- Allocation:
  - Fixed or dynamic size.

---

17. Space Management

- Techniques:
  - Compression: Reduces file size.
  - Deduplication: Eliminates redundant copies.
  - Hierarchical storage management: Moves less-used data to slower storage tiers.